

AI for Economic Research: Dynamic Models, Language, and Agents

Lecture 6.2b: Global Solution via Deep Neural Networks

Zhigang Feng

Spring 2026

Topic 6.2b Agenda

1. **Global Solution via DNN** — full distribution as state, value/policy networks

*Arc: T1 Aiyagari + computation → T2a Why ML / Euler → **T2b Global DNN** → T3 KS / DeepHAM → T4 DEQN / OLG → T5 MFG → Lab06*

Global Solution via DNN

Why (deep) RL helps

Three reasons NN-based methods outperform grid methods on HA models:

1. **Flexible function approximation.**

No structured Cartesian grid; networks handle high-dimensional inputs gracefully.

2. **Sampling-based training.**

RL methods were designed for unknown environments; here we re-use the data-generation tools (rollouts, replay, importance sampling) even though our model is known. This avoids constructing exhaustive grids or closed-form distribution evolution.

3. **Hardware acceleration.**

Forward/backward passes through neural networks are highly parallel matrix operations that GPUs and TPUs execute efficiently. Grid VFI is much harder to parallelise.

Global solution: full distribution as part of the state

- In Aiyagari we exploited *stationarity*: the distribution λ was a fixed object and dropped out of the household state.
- Once we add aggregate shocks, λ_t becomes part of the dynamic state. The household problem is

$$V(a, z, \lambda) = \max_{c, a'} \{u(c) + \beta \mathbb{E}[V(a', z', \lambda') \mid z]\}.$$

- The law of motion for λ is

$$\lambda' = G(\lambda, Z, Z'),$$

itself induced by the policy $a'(\cdot, \cdot, \lambda)$ and the shock kernel.

- λ is an infinite-dimensional object — this is the heart of the curse of dimensionality in HA models with aggregate shocks.

Household problem with an evolving distribution

At the individual level, with state (a, z, λ) :

$$V(a, z, \lambda) = \max_{a'} \{u((1 + r(\lambda))a + w(\lambda)z - a') + \beta \mathbb{E}[V(a', z', \lambda') | z]\}$$

with $\lambda' = G(\lambda, Z, Z')$.

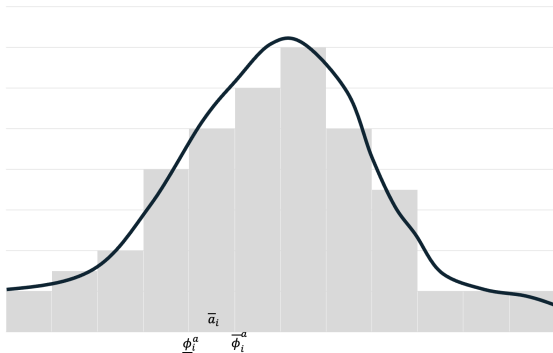
- Two challenges:

1. How to represent λ compactly enough that a NN can take it as input.
2. How to evolve λ forward without simulating $\sim 10^6$ agents.

- Three families of solutions:

- *Moment approximations* (Krusell–Smith): summarise λ by a few moments.
- *Generalised moments* (DeepHAM): let the network learn the right summary statistics.
- *Histogram transport* (DEQN): keep the full discretised distribution.

Approximating the distribution as a NN input



- One option: discretise λ on a fixed grid, feed the histogram heights to the NN.
- Another: pass a finite vector of moments (means, variances, percentiles).
- DeepHAM: *learn* basis functions $Q_1, \dots, Q_J$ and feed $\frac{1}{N} \sum_i Q_j(a_i)$ as moments.

DNN representation of V and the policy

Parameterise

$$V(S; \theta^V), \quad c(S; \theta^c), \quad S = (a, z, \lambda).$$

- Each network is a multilayer perceptron with parameters $\theta = (\text{weights, biases})$.
- Inputs: individual state (a, z) + summary of λ (moments or generalised moments).
- Outputs: scalar V , scalar consumption (or savings rate).
- Activations: ReLU, GELU, Mish — all popular for HA-style problems.
- Initialisation: Xavier/Glorot or Kaiming, depending on the activation.

Update value function via simulation + supervised learning

Given current policy $f_{\sigma}^{(n)}$, generate N_V initial states $\{S_{i,0}\}_{i=1}^{N_V}$ and simulate T_V periods forward. Compute the empirical (truncated) discounted utility:

$$\hat{V}(S_{i,0}) = \sum_{t=0}^{T_V-1} \beta^t u(c_{i,t}).$$

Train the value network by supervised regression:

$$\theta^{V,(n+1)} \in \arg \min_{\theta^V} \sum_{i=1}^{N_V} (V(S_{i,0}; \theta^V) - \hat{V}(S_{i,0}))^2.$$

- This is sometimes called *Monte Carlo value estimation*.
- Larger T_V reduces bias but increases variance and cost.

Update policy with terminal value function

Given the just-updated value network $V(\cdot; \theta^{V,(n+1)})$, choose policy parameters to maximise the truncated objective with terminal value:

$$\theta^{c,(n+1)} \in \arg \max_{\theta^c} \mathbb{E} \left[\sum_{t=0}^{T_g-1} \beta^t u(c_{i,t}) + \beta^{T_g} V(S_{i,T_g}; \theta^{V,(n+1)}) \right]$$

subject to the budget constraint

$$c_{i,t} = (1 + r(\lambda_t)) a_{i,t} + w(\lambda_t) z_{i,t} - a'_{i,t}, \quad a'_{i,t} = c(S_{i,t}; \theta^c).$$

This is a long-horizon *differentiable rollout*: gradients propagate through the simulated path.

Transition function G as a matrix

On a discretised asset grid $\{a_1, \dots, a_{N_a}\}$, the policy $a'(\cdot)$ together with the shock kernel induces a Markov matrix

$$\hat{G}_{i,j} = \Pr(a' \in \phi_j^a \mid a \in \phi_i^a),$$

where ϕ_j^a are the discretisation cells.

- $\hat{G}$ has $O(N_a^2)$ entries; sparse if the policy moves between adjacent cells.
- Updating λ one period: $\lambda^{(t+1)} = \hat{G}^\top \lambda^{(t)}$.
- The same matrix is used in classical Aiyagari computation; here we re-use it inside the differentiable training loop.

Conditional and unconditional transition kernels

Unconditional:

$$\lambda'(z' \in Z', a' \in A') = \int_{Z \times A} \mathbb{K}\{a'(a, z, \lambda) \in A', z'(a, z, \lambda) \in Z'\} d\lambda(z, a).$$

Conditional (given current (a, z)):

$$\lambda'(z' \in Z', a' \in A' \mid z \in Z, a \in A) = \int_{Z \times A} \mathbb{K}\{\cdots\} d\lambda(z, a) / \lambda(Z \times A).$$

- Both kernels live in the Wasserstein space of probability measures on S .
- The induced *stochastic process on distributions* is the analytical object that DeepHAM and DEQN approximate by neural networks.

Discretised transition: i.i.d. shocks, monotone policy

When z is i.i.d. and the policy $a'(a, z)$ is monotone in a for each z , we can solve for transition cell boundaries explicitly:

$$\hat{G}_{i,j}^{\lambda, \eta_x} = \Pr(\phi_j^a \ni a_+ \leq \bar{\phi}_j^a \mid a \in \phi_i^a) = F_z(\bar{z}_{i,j}) - F_z(\underline{z}_{i,j}),$$

where $(\underline{z}_{i,j}, \bar{z}_{i,j})$ are the income shock thresholds that map ϕ_i^a to the boundaries of ϕ_j^a under the policy.

- Closed-form bin probabilities $\Rightarrow$ no Monte Carlo noise in the λ -update.
- Differentiability is delicate: the indicator functions inside the kernel are non-smooth, and need to be replaced by smooth approximations (e.g. sigmoids with sharp slopes) for gradient propagation.

Next (T3): Krusell–Smith with aggregate uncertainty, and DeepHAM as the deep-learning solution.